

DriverAI Hiring Agent Phase 1 - Consolidated Technical Design Document

Version 3.0 | September 8, 2026 | Supersedes v2.3 (original) and v2.4 (reviewed)

This document merges every surviving P1 source into one specification. It folds the original 16-page TDD, the September 8 reviewed edition, the operations README, the detailed architectural summary and the verified reply catalogue into a single text, and adds the material none of them carried. Every structural claim below was re-verified this session against the supplied deployment package. Nothing in the running tenant was imported, modified, restarted or mailed.

0. Provenance, evidence boundary and status legend

Source document	Version and date	Disposition in this merge
P1_Technical_Design_Document.md / .pdf	v2.3, 8 Sep 2026, 16 pages	Full operational narrative retained; nine claims corrected
P1_Technical_Design_Document_Reviewed.md	v2.4, 8 Sep 2026	Findings, verified reply bodies and acceptance list retained in full
docs/README.md	24 Aug 2026, 84 KB	Config reference, observability and replay tooling merged in; three stale counts corrected
docs/p1_detailed_summary.md	8 Sep 2026, 34 KB	Incident history, cap rationale and open-item register merged in
docs/MAIL_REPLIES.md and MAIL_REPLIES_REVIEWED.md	4 Aug / 8 Sep 2026	Superseded by section 7 of this document
CURRENT_ARCHITECTURE.md	8 Sep 2026	P1/P2 boundary statements merged into section 2
DriverAI-Hiring-AutoReply-apply.zip	8 Sep 2026, 19,900 bytes	Primary evidence; re-parsed in full for this merge

The supplied package and both workspace copies share one SHA-256:

5fa60a5941bc60fbed3c41603169908684dd277bb77df10502f956888bb62711

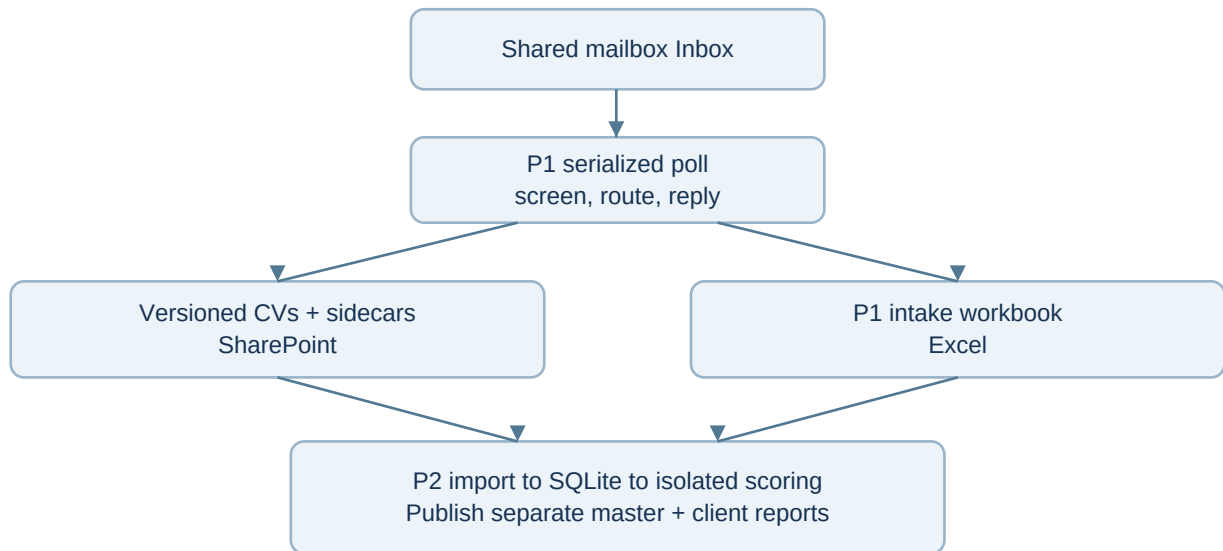
Status markers used throughout. **Verified** means re-read out of that package this session. **Corrected** means at least one source document stated it wrongly and the correct value is given. **Open** means a defect that still exists in the shipped artifact. **Proposed** means design work that has not been built.

Evidence surface	Observed state
Supplied ZIP definition	131 actions, 18 at top level, maximum nesting depth 8 of a platform limit of 8
Applicant mail in the ZIP	All six sends enabled, addressed to the incoming sender, BCC to the admin address
Local flow_config.json	send_applicant_emails false, send_admin_failure_alerts true
Local loose definition.json	The same six sends replaced by Compose suppression markers
Connection references	shared_office365, shared_sharepointonline, shared_excelonlinebusiness
Active cloud flow	Not inspected. The owner reports it running in test mode

Those states coexist because a configuration change reaches the tenant only after a rebuild and a manual import. Testing this document required no change to the current suppression or routing.

1. Executive summary and the four-role model

P1 is a scheduled Power Automate flow that acts as the automated gateway for everything arriving at `apply@driverai.io`. It screens, classifies, stores and acknowledges inbound hiring mail, then hands a clean queue to the Phase 2 scoring engine. It performs no scoring, no interview scheduling and no rejection mail of its own.



The flow is easiest to reason about as four sequential roles.

The Gatekeeper decides whether a message is safe and genuine. Three substring gates screen sender, subject and content. A blocked message is moved unread to Junk Email and the run terminates.

The Registrar stores what survives. It guarantees the dated folder exists, saves each accepted attachment under a fresh GUID name, writes a JSON metadata sidecar beside it, and either appends a new candidate row or patches the matched existing row.

The Receptionist answers the applicant and clears the queue. It sends one of six templates when applicant mail is enabled, stamps the outcome into the row, then marks the message read and moves it to Archive.

The Handoff ends P1's involvement. A row carrying `Status = New Email Received` is P2's signal to score. The two phases never call each other.

1.1 The six data storage layers

Storage layer	Cloud location	Operational role
Work queue	<code>apply@driverai.io</code> Inbox	The active to-do list. Mail here is unhandled work
File vault	SharePoint /Shared Documents/Candidate_Resumes/	Permanent resume storage, foldered by year and month
Sidecar manifest	SharePoint intake_*.json beside each CV	Machine-readable intake metadata paired to the file
Candidate index	Excel /Master_Files/Sharepoint_Master_File.xlsx	The 33-column roster shared with Phase 2
Archive vault	<code>apply@driverai.io</code> Archive	Processed mail, read on success and unread on failure
Quarantine bin	<code>apply@driverai.io</code> Junk Email	Screened mail, always left unread

2. Architecture, ownership and the Phase 2 boundary

P1 owns mailbox intake and the Excel intake workbook. P2 owns extraction, scoring, geography filtering and client reporting. The workbook is the entire contract between them, and neither phase invokes the other.

Functional area	Phase 1 ownership	Phase 2 ownership
Inbox	Polling, screening, marking read, moving to Archive or Junk	No mailbox access at all
Files	Saves accepted PDF and DOCX attachments plus sidecars	Reads stored resumes for text extraction and scoring
Workbook	Creates intake rows; patches status, update counter, update time and Mail Sent	Writes scored profile fields and report state
Applicant mail	Six intake templates listed in section 7	Decline and missing-information mail, sent through Graph directly
Sheet order	Appends to the physical bottom in arrival order only	Owns re-sorting and month or year divider rows

Corrected. The original TDD described the split as 11 P1 columns against 22 P2 columns. That is close but not exact, because Mail Sent is written by both phases and several sidecar values never reach the workbook. Ownership is per operation, not per column. Section 9 gives the verified breakdown.

Corrected. Older architecture notes described the P2 SQLite path as unimplemented. It exists in `local_pipeline.py` and `sqlite_store.py` today, covering incremental import, a worker lock, isolated scoring, version-checked completion and publication. P2 publishes to its own master report and client workbook, with explicit guards against overwriting P1's intake workbook. P1's workbook is an intake index, not the complete scored database.

Open. `LocalClient.send_mail` returns False and the parent SQLite pipeline has no delivery dispatcher. Clearing P2's suppression flag alone does not produce live P2 applicant mail. This is a P2 gap recorded here only because it changes what the combined system does after P1 hands off.

3. Trigger, queue and real capacity

The trigger is a Recurrence firing every minute with `concurrency.runs = 1`. `Get_unread_emails` then reads the Inbox with `top = 1`, `fetchOnlyUnread = false` and `includeAttachments = true`.

Read and unread mail are both eligible, which is deliberate. A recruiter who previews a message in Outlook marks it read; a trigger keyed to the unread flag would drop that candidate permanently. The physical Inbox folder is the queue instead, so anything sitting in it is unhandled work.

Terminal transitions are fixed. Success marks the message read and moves it to Archive. A screened message stays unread and moves to Junk Email. A processing failure leaves it unread and moves it to Archive, where the unread flag distinguishes it from handled mail.

Corrected. The original TDD presented one message per minute as a guaranteed 60-second service level and described the design as eliminating Excel write conflicts entirely. Neither holds. Serialized polling with one message per run is a nominal capacity of about one message per minute, and backlog, long-running actions and platform outages all extend it. No oldest-first ordering is configured, so queue order is whatever the connector returns. The concurrency limit serializes P1 against itself and nothing else. Microsoft documents that the Excel Online (Business) connector does not support simultaneous modification from different clients, so a person editing the workbook, or any other writer, remains a genuine conflict source. Reference:

<https://learn.microsoft.com/en-us/connectors/excelonlinebusiness/>

Note. `trigger.interval_min` reached its current value of 1 by an owner instruction on 14 August 2026, after a spell at 2 for request-quota headroom. One minute is 1,440 runs per day against the owner plan, which is worth watching against the Power Platform daily request budget.

4. Screening gates

Three gates run in order. Each is a substring test, and each is disabled outright for an allow-listed sender.

Gate	Action	List size	Scanned field	Failure action
1. Sender and self-loop	IsSystem	43 fragments	Lowercased raw From header	Junk Email, unread, terminate
2. Subject	IsSubject	27 strings	Lowercased subject	Junk Email, unread, terminate
3. Content and malware	IsSpam	123 phrases	Lowercased subject and body	Junk Email, unread, terminate
3b. Link shorteners	MatchLinkShortener	12 domains	Same fields, only when no accepted resume is attached	Folded into gate 3

Gate 1 includes `apply@driverai.io` itself, which is what stops the mailbox replying to its own messages in a loop.

Verified mechanism, previously undocumented. `LowerBody` is not the message body alone. It is the body concatenated with a space, then every attachment filename joined by spaces, then a trailing space. That single composition is why attachment names are screened at all, and why the malware group can anchor each extension with a trailing space. `.exe` matches an attachment called `payload.exe` because the join guarantees a following space, while `iso.org` in prose no longer trips `.iso`. That anchoring came out of the 31 July 2026 false-positive audit.

Corrected. The original TDD broke the 123 content phrases into four groups of 30 scam, 36 offensive, 20 malware and 25 foreign-language terms. Those sum to 111. The shipped list carries a fifth group of 12 vendor and outsourcing pitch phrases, ending with entries such as `software development agency` and `development partner`, which no source document described. The 12 link shorteners are a separate list and are not part of the 123.

Corrected. The README config table records 32 bad senders and 24 bad subjects. The shipped package carries 43 and 27.

Open, high severity. The tester allow-list is a substring test. `MatchAllowSender` evaluates `contains(LowerFrom, item())` against the two entries `yashv@driverai.io` and `tracys@driverai.io`, and `LowerFrom` is the raw lowercased From header including any display name. An address such as `yashv@driverai.io.example.net`, or a display name containing that text, satisfies it. All three gate expressions carry `length(MatchAllowSender) equals 0` as a conjunct, so one false match disables sender screening, subject screening, content screening and the self-loop guard at once. It also switches the reference lookup in section 5 from sender matching to Application ID matching, which is the more dangerous half. The fix is to extract the address, normalize it, and compare for equality, keeping tenant authentication and tester privilege as separate checks. Evidence: `flow/build_zip.py` lines 449-466 and 1674-1679; a local expression-equivalent probe reproduces the false match, and no external message was sent.

Open. Screening is substring matching only. There is no antivirus scan, no MIME or magic-byte validation, no attachment size or count limit, and no authenticated sender verification. A file named `resume.pdf` that is actually an executable reaches SharePoint, where Microsoft 365's own scanning is the only backstop. Hardening this needs new actions and probably a new connector or function, so it is a design decision rather than a config change.

Proposed. Document the maximum attachment count and size, the behaviour on corrupt or password-protected files, preview truncation, the backlog threshold and the shared-mailbox quota. The shipped flow establishes none of these limits.

5. Routing, identity and duplicate detection

A message that clears all three gates enters classification. HasAppRefGate is evaluated before new-application intake, so a reply quoting a reference is never ingested as a fresh duplicate row.

Route	Conditions	Reply	Workbook effect
1. New intake	Accepted resume, sender not seen inside 90 days	Acknowledgment	Add_row appends at row 3 or below
2. Duplicate	Accepted resume, sender already on file inside 90 days	Duplicate notice	Patch_dup_attempts on the matched row
3. Missing resume	No attachments, application keywords present	CV request	None
4. Wrong format	Attachments present, none accepted	Format request	None
5. Resume update	Reference quoted, sender resolves, accepted resume attached	Update acknowledgment	Patch_update_attempts
6. Follow-up note	Reference quoted, sender resolves, no accepted resume	Noted reply	Patch_followup_attempts

Reference detection scans subject then body for the literal app-20 and lifts a 22-character token, which keeps it valid to 2099.

Verified identity rule, stated more precisely than the original. Get_rows_ref builds its filter two ways. For an ordinary external sender it filters on the normalized sender address, so quoting somebody else's reference cannot select their row. For an allow-listed tester with a quoted reference it filters on Application ID directly. The original TDD described only the first behaviour and called it a fake-reference guard. That guard is real for external mail, and it is bypassed entirely by an allow-list match, which is why the substring defect in section 4 matters beyond spam.

Verified. Duplicate eligibility is order-independent. Submitted_ticks projects every matching row to a tick value and Is_duplicate compares max(ticks) against the 90-day window, so it survives P2 re-sorting the sheet newest-first. This replaced a last(Filter_submitted) read that broke on 3 August 2026 exactly that way, letting a returning applicant whose first contact predated the window read as brand new.

Open, medium severity. The decision is order-independent but the target is not. Having decided on max(ticks), every downstream action patches first(...) of the same result set, and the reference path likewise takes first(Get_rows_ref). A sender holding several applications can therefore have an older record updated by a newer message. Specify which application wins, then apply that same selection to the decision and the patch. Evidence: Submitted_ticks, Is_duplicate, first(body('Filter_submitted')), Get_rows_ref and AppRef_current in the generated definition.

Verified. Sender matching normalizes on both sides. The \$filter and the stored Email column both lowercase the address and strip a Name <addr> wrapper. This is forward-only by design: rows written before the change keep their raw value, and no backfill was performed.

5.1 Contact caps and silent intake

Three counters share one workbook column, Application Updates, and two thresholds.

Existing count	Duplicate, update and follow-up behaviour
0 or 1	Row and file updated, normal reply sent
2	Row and file updated, final-notice variant sent

Existing count	Duplicate, update and follow-up behaviour
3 or 4	Row and file updated, no mail at all
5 or more	Capped path stops; nothing is saved on that route

The two-tier design is deliberate. `reply_cap = 3` limits how many contacts get mail, while `duplicate_notice_max`, `update_resume_max` and `followup_reply_max` at 5 limit how many are recorded. Between the two, the resume and the row stay current so P2 always scores the newest document even after the applicant stops hearing back.

Corrected. The original TDD said the final-notice swap fires when a candidate "reaches contact 3". The shipped expression swaps when the existing count is already 2 or greater, which is the third contact counting from zero. The wording invited an off-by-one reading.

Open. The counter records attempts, not deliveries. A suppressed send and a failed send both consume a contact, so the column cannot be read as a count of mail the applicant received. The initial acknowledgment, CV request and format request do not share this limiter at all.

5.2 State transitions

Input	Persisted effect	Reply and terminal handling
New valid resume	Versioned file, sidecar, new row, status New Email Received	Acknowledgment when enabled; read, moved to Archive
Duplicate inside 90 days	Versioned file and sidecar, existing row requeued below cap	Duplicate notice below reply cap
Reference plus resume	Versioned file and sidecar, matched row requeued below cap	Update acknowledgment below reply cap
Reference, text only	Counter and Last Updated Date only; reply body not stored	Noted reply below reply cap
Missing resume or wrong format	No candidate row	CV or format request when enabled
Screened or system mail	No intake record	No applicant reply; unread in Junk Email

An empty Inbox means mail left the queue. It does not mean every application was stored or scored. Replaying an archived message can create another file and consume another contact count, because there is no durable source-message replay key.

6. Execution sequence and verified action inventory

1. Poll_unread_shared_mailbox fires on the one-minute recurrence with concurrency 1.
2. Get_unread_emails reads one Inbox message with attachments. Has_unread_email terminates the run when nothing is waiting.
3. CurrentEmail and CONFIG compose the message payload and the static SharePoint and Excel parameters.
4. LowerFrom, LowerSubject, AttachNames and LowerBody normalize the scan fields. AllowSenders, MatchAllowSender and QuotedRef establish tester status and any quoted reference.
5. Gates run in order as IsSystem, IsSubject, IsSpam, each moving a blocked message to Junk Email and terminating.
6. HasAppRefGate runs next. When a reference is present, Get_rows_ref and IsKnownSender resolve the application, then Has_resume_in_update splits route 5 from route 6. Both branches patch the row, mark read, move to Archive and terminate.
7. ResumeFilesAll, ResumeFilesKept and ResumeFiles select accepted attachments. StoredResumeFiles assigns each a cv_<guid>.<ext> name and ResumeManifest builds the sidecar file list.
8. HasResume splits the remainder. With no accepted file it runs Has_application_keyword for the CV request, HasWrongFormat for the format request, or Notify_ignored_mail for genuine non-application mail.
9. With an accepted file, Is_duplicate chooses between the duplicate branch and new intake. New intake runs Ensure_resume_folder, Save_resumes_to_SharePoint, Capture_Create_file, Preserve_Create_file, Add_row, Send_acknowledgment, Patch_mail_sent and Ack_stamp_done.
10. Mark_as_read then Move_to_processed clear the queue. On a HasResume failure, Notify_failure fires and Move_failed_to_archive_unread parks the message unread in Archive.

Structural property	Verified value
Total actions in the package	131
Top-level actions	18
Maximum nesting depth	8 of a platform maximum of 8
Applicant send actions	6, all enabled in the supplied ZIP
Admin notification actions	14
Excel operations	Get_rows, Get_rows_ref, Add_row and 7 patch actions
Connection references	3
flowFailureAlertSubscribed	true

Open. The build reports zero nesting headroom and names Capture_Create_update_file, Create_update_file, Mark_as_read_followup and Mark_as_read_followup_noreply as the branches sitting at the ceiling. Any new If, Foreach or Scope inside those branches fails import. This is why the seven patch-failure alerts were wired as siblings rather than children, and it constrains every future change to the update and follow-up paths.

7. Applicant reply catalogue

Source: the six enabled actions in the supplied ZIP, cross-checked against section 15 of flow/build_zip.py. P2 config.yaml supplies none of this copy. Text below is the rendered wording with {AppRef} standing in for the dynamic reference and HTML styling omitted. These are existing templates, not newly deployed messages.

All six use SharedMailboxSendEmailV2, sending from apply@driverai.io, addressed to the incoming CurrentEmail.from, with BCC to yashv@driverai.io. They are new outbound messages carrying a reference in the subject, not connector reply-in-thread operations. No Reply-To header and no original-message thread identifier is set. The imported connection's real permissions and sender behaviour still need tenant verification.

The local suppressed build replaces all six with Compose actions carrying a suppression marker. The supplied ZIP enables all six. Its BCC copies the admin address; it does not redirect the primary recipient away from the applicant. Finding 15.2 concerns the updated-resume acknowledgment in 7.5 specifically.

7.1 Application acknowledgment

Action: Send_acknowledgment.

Trigger and data effect: New applicant with an accepted resume; send follows successful Add_row. A new intake row is created.

Subject: Application Received - DriverAI (Ref: {AppRef})

Hello,

Thanks for applying to DriverAI. Your application has been received and is under review.

Reference number: {AppRef}

If you are selected, a member of our team will contact you to discuss next steps. You do not need to reply unless you are updating your resume. To do that, send a new or reply email to apply@driverai.io with the subject "Update - {AppRef}" and attach the new file.

Good luck on your next journey.

Warm regards,
The DriverAI Recruiting Team

This is an auto-generated email and this mailbox is not monitored.

7.2 Duplicate notice

Action: Send_duplicate_notice.

Trigger and data effect: Existing sender inside the 90-day window, no reference route, accepted resume, and shared counter below reply_cap. Row is queued and counter advances.

Subject: Your DriverAI Application Is Already On File (Ref: {AppRef})

Hello,

Your application is already on file with DriverAI and under review, so there's no need to resubmit.

If you are selected, a member of our team will contact you to discuss next steps. You do not need to reply unless you are updating your resume. To do that, send a new or reply email to apply@driverai.io with the subject "Update - {AppRef}" and attach the new file.

Good luck on your next journey.

Warm regards,

The DriverAI Recruiting Team

This is an auto-generated email and this mailbox is not monitored.

7.3 Missing resume

Action: Send_CV_request.

Trigger and data effect: Application keywords but no accepted attachment in the missing-CV path. No candidate row is created.

Subject: Please Attach Your Resume - DriverAI

Hello,

Thanks for your interest in DriverAI. We didn't see a resume attached, so your application isn't complete yet.

Please reply with your resume attached in PDF or Word (.docx) format. Once we have it, your application goes under review, and if you are selected, a member of our team will contact you to discuss next steps.

Good luck on your next journey.

Warm regards,
The DriverAI Recruiting Team

This is an auto-generated email and this mailbox is not monitored.

7.4 Unsupported attachment

Action: Send_wrong_format.

Trigger and data effect: Attachments fail the accepted-resume selection path. No candidate row is created. This is an extension/selection check, not proof the file was opened.

Subject: Please Resend Your Resume as PDF or Word - DriverAI

Hello,

Thanks for your interest in DriverAI. We received your submission, but we couldn't open the attached file.

Please reply with your resume as a PDF (.pdf) or Word (.docx) file and we'll process it right away. If you are selected, a member of our team will contact you to discuss next steps.

Good luck on your next journey.

Warm regards,
The DriverAI Recruiting Team

This is an auto-generated email and this mailbox is not monitored.

7.5 Updated resume acknowledgment

Action: Send_update_ack.

Trigger and data effect: Reference-bearing mail resolves to an existing sender/application and includes an accepted resume; shared counter below reply_cap. See finding 15.2, which concerns this message specifically.

Subject: Updated Resume Received - DriverAI (Ref: {AppRef})

Hello,

Thanks for sending your updated resume. Your application now reflects the latest version and our team will review it.

If you are selected, a member of our team will contact you to discuss next steps. You do not need to reply unless you are updating your resume again. To do that, send a new or reply email to apply@driverai.io with the subject "Update - {AppRef}" and attach the new file.

Good luck on your next journey.

Warm regards,
The DriverAI Recruiting Team

This is an auto-generated email and this mailbox is not monitored.

7.6 Text-only follow-up

Action: Send_noted_reply.

Trigger and data effect: Reference-bearing mail resolves to an existing sender/application without an accepted resume; shared counter below reply_cap. Counter and Last Updated Date change; reply body is not added to the candidate row.

Subject: Message Received - DriverAI (Ref: {AppRef})

Hello,

Thanks for following up. Your message has been noted alongside your application.

If you are selected, a member of our team will contact you to discuss next steps. You do not need to reply unless you are updating your resume. To send one, reply with the subject "Update - {AppRef}" and attach the file (PDF or Word).

Good luck on your next journey.

Warm regards,
The DriverAI Recruiting Team

This is an auto-generated email and this mailbox is not monitored.

7.7 Final notice, caps and copy defects

For Duplicate notice, Updated resume acknowledgment and Text-only follow-up, the normal paragraph beginning "If you are selected" is replaced on the third counted follow-up by:

This is our final automated reply regarding this application. If there is a match, a member of our team will contact you directly. You do not need to reply, and you are welcome to apply again after 90 days.

All three paths share Application Updates. Existing count 0 or 1 uses normal copy; count 2 uses final notice; counts 3 and 4 process silently; count 5 or greater stops the capped update/follow-up path. Suppression and unsuccessful sends can still consume this shared contact counter. It is not a count of successfully delivered emails. The initial acknowledgment, missing-CV request and wrong-format request do not share this limiter.

The current footer says the mailbox is not monitored while the body invites replies. Proposed replacement, not deployed: "This is an automated confirmation. Replies are processed by our hiring system; a recruiter will contact you if further information is needed."

The follow-up wording says a message is noted alongside the application, but the candidate record does not retain its body. Either persist the reply and make it available to review, or narrow the copy to "We received your

follow-up email." Do not promise a review time: none is implemented or stated in the current templates.

8. Admin alerting, delivery audit and observability

The package contains 14 notification actions: 13 failure alerts and one informational notice. All are addressed directly to yashv@driverai.io and all retry exponentially twice at ten-second intervals.

Alert action	Watched action	Consequence when it fires	Recovery
Notify_poll_failure	Get_unread_emails	Mailbox connection dropped or throttled	Check the Outlook connection in Power Automate
Notify_failure	HasResume	Unhandled error during candidate processing	Read the run history, then replay the message
Notify_add_row_failed	Add_row	No candidate row exists anywhere	Move the mail from Archive to Inbox as unread
Notify_create_file_failed	Create_file	Row exists, resume file never saved	Move the mail from Archive to Inbox as unread
Notify_create_dup_update_file_failed	Create_dup_update_file	Repeat applicant's new resume not saved	Move the mail from Archive to Inbox as unread
Notify_create_update_file_failed	Create_update_file	Reference update's new resume not saved	Move the mail from Archive to Inbox as unread
Notify_mail_sent_failed	Patch_mail_sent	Reply sent, audit cell not stamped	Verify the Mail Sent cell by hand
Notify_dup_attempts_failed	Patch_dup_attempts	Duplicate counter not incremented	Check and correct the row
Notify_dup_attempts_noreply_failed	Patch_dup_attempts_noreply	Past-cap duplicate counter not incremented	Check and correct the row
Notify_update_attempts_failed	Patch_update_attempts	Update counter not incremented	Check and correct the row
Notify_update_attempts_noreply_failed	Patch_update_attempts_noreply	Past-cap update counter not incremented	Check and correct the row
Notify_followup_attempts_failed	Patch_followup_attempts	Follow-up counter not incremented	Check and correct the row
Notify_followup_attempts_noreply_failed	Patch_followup_attempts_noreply	Past-cap follow-up counter not incremented	Check and correct the row
Notify_ignored_mail	Non-application mail archived	Informational only, low importance	None required

Corrected. The README describes nine retrying alerts and two intake-write alerts. The shipped package has 14 notifications, of which four are intake-write watchers covering Add_row, Create_file,

Create_dup_update_file and Create_update_file.

Verified design note. Six of the seven patch alerts watch genuinely silent failures where the alert is the only signal, because the enclosing If reports Succeeded and the branch ends on a Terminate with runStatus: Succeeded. Patch_dup_attempts_noreply is the exception: it is the last action in its scope, so its failure propagates to HasResume, turns the run red and fires Notify_failure as well. That duplicate is deliberate and its body says so, because the reply-path twin is swallowed and appending any sibling to that scope would start hiding the failure.

8.1 Observability, retention and blind spots

Neither TDD carried this table. It comes from the operations README and is reproduced here because it defines what evidence survives an incident.

Signal	Where it lives	Retention
Per-run step detail	Power Automate run history	30 days, platform default
New candidate	Workbook row plus saved resume and sidecar	Permanent
Per-candidate audit trail	Application ID, Received Date, Status, Mail Sent, Application Updates	Permanent
Processing and patch failures	The 13 admin alert emails	Admin inbox
Platform-level failure backup	flowFailureAlertSubscribed	Power Automate alert mail
Volume metrics, spam counts, reply-type counts	None exists	Not collected
Success notification	None exists	Not collected

There is no custom log table and no counters. Counting applicants for a period means filtering the workbook by Received Date. Counting screened mail means counting terminated runs in the run history.

Blind spot one. A screened message creates no row, increments no counter and raises no alert, by design. A false positive is therefore invisible unless somebody opens the Junk Email folder. The mitigation is procedural: replay the phrase lists over recent mail after any change to those lists. The 31 July 2026 replay covered all 167 terms against 278 messages and found exactly one term that had ever fired, marketing@, on three emails from one genuine applicant. Probing then proved seven terms unsafe, including alert, invoice, receipt, payment, survey and the unanchored .exe and .iso against exeter.ac.uk and iso.org. After the fix the same replay produced zero hits. Three of the six Junk messages had matched no P1 gate at all; Exchange Online junked those before the flow ran, which P1 cannot influence.

Blind spot two. Run history is the only step-level log and it is not permanent. Once it ages out, a failed run's exact error is gone and the alert emails are the only durable record. That is why they retry.

8.2 Delivery audit contract

Mail Sent is a last-status cell, not a delivery ledger. Successful connector execution is not proof of inbox delivery, and bounces and ambiguous timeouts need reconciliation that nothing currently performs. A green run and an empty Inbox together are not a health signal.

Proposed. A durable outbox with one record per attempt, carrying message key, application ID, template version, intended recipient, actual recipient, test or live mode, a queued, sent, failed or unknown status, provider message ID, attempt time and error text. No such outbox exists in the reviewed flow.

9. Data contracts

9.1 The workbook

The workbook is `/Master_Files/Sharepoint_Master_File.xlsx`. Row 1 holds the canonical headers, row 2 is a permanent blank spacer, and candidate records begin at row 3. The shipped setup template `P1_Templates/HiringAgent_P1_CandidateList.xlsx` was re-read for this merge and carries two tables.

Table	Sheet role	Column count	Table range
HiringAgent_P1_Candidates	Main candidate roster	33	A1:AG2
HiringAgent_P2_Rejected	Rejected roster, shipped with the template	32	A1:AF2

Corrected. The README states 30 main columns and 29 rejected columns. The shipped template has 33 and 32. The original TDD's 33-column list is correct and is retained below. The Rejected sheet is not created by P2 at runtime; it ships inside the template, and its 32nd column is `Decline Sent` rather than the main sheet's `Mail Sent` and `Info Request Sent` pair.

`Add_row` writes exactly 11 fields and omits the rest, so they land as genuinely blank cells. That matches the real tenant export format and avoids the import-time schema binding that was dropping fields in the designer.

Column	Name	Written by P1 at intake
1	Application ID	APP-yyyyMMdd-HHmm plus 4 characters derived from the message id
2	Received Date	Mountain Standard Time, UTC minus 7
3	Last Updated Date	Mountain Standard Time, UTC minus 7
6	Full Name	Display name from the From header, or the local part with dots and underscores as spaces
7	Email	Normalized lowercase sender address
20	Mail Subject	Raw subject
21	Mail Body	bodyPreview, roughly 255 characters
22	Status	New Email Received
23	Has Resume	Yes
24	Original Filename	Original attachment names, comma separated
25	Application Updates	0

Columns 4, 5, 8 through 19, 26 through 31 and 33 are Phase 2 territory: Category, Resume Link, Phone, Location, Country, Years Exp, Current Skills, Education, Education Start and End Date, Looking For Role, Suggested Role 1 to 3, Retry Count, Portfolio 1 to 3, Resume URL, Resume Folder Path and Info Request Sent. Column 32, Mail Sent, is shared: P1 stamps it after a send path succeeds or is suppressed, and P2 uses it for its own mail.

Two stability rules survive from the original document and both still hold. Excel Online fails with HTTP 409 `InsertDeleteConflict` if column filters are active when a row is added, so header filters must stay off. P1 only ever appends at the physical bottom in arrival order, and never inserts separator rows, because out-of-order arrivals would scatter blanks through the table. Sorting and dividers belong to P2's `resort_candidate_sheets.py`, which runs on demand and is not wired into this flow.

9.2 The sidecar

Each accepted attachment gets a companion JSON file named `intake_<stored name>.json` written into the same dated folder immediately after the upload.

Field	Value on new intake	Value on a reference update
Application ID	Newly minted AppRef	Matched row's Application ID
Received Date	Message received time, UTC minus 7	Matched row's Received Date
Last Updated Date	Same as Received Date	This message's received time
Application Updates	0	Existing count plus 1
Email	Normalized sender	Normalized sender
Full Name	Empty string, left for extraction	Matched row's Full Name
Original Filename	Joined original attachment names	Joined original attachment names
Resume URL	manifest: followed by a JSON array of name, original_name and folder	Same
Resume Folder Path	/Candidate_Resumes/YYYY/MMMM	Same
Mail Subject and Mail Body	Subject and bodyPreview	Subject and bodyPreview
Status and Has Resume	New Email Received, Yes	New Email Received, Yes

Verified. Three different folder conventions coexist and all three are correct for their caller. `CreateFile` takes the library-qualified `/Shared Documents/Candidate_Resumes/...`. `CreateNewFolder` takes a path relative to the library root, because its `table` parameter already names the library. The manifest stores the Graph-drive-relative `/Candidate_Resumes/...`, because P2 reads it through the Graph drive API.

Open. Uploads precede sidecar creation, and the Excel write is a third separate operation. There is no transaction across the three. Before a sidecar can be treated as a complete intake event it needs a schema version, a stable source-message identifier, an attachment identifier or hash, an ingestion timestamp, a run correlation identifier and a reconciliation status. The document must also define how a multi-attachment group becomes one committed event and how to recover a file whose sidecar is missing.

10. Storage and file naming

Resumes live under `Shared Documents/Candidate_Resumes/<Year>/<Month>/`. Every dated folder is created explicitly by `Ensure*_resume_folder` before any save, never assumed. The folder action is a safe no-op when the folder exists, and the following save runs whether it succeeded or failed.

Verified. The reliability overlay in `flow/reliability_flow.py` selects one GUID per incoming attachment and every save path uses it. `StoredResumeFiles` computes `cv_<guid>.<extension>` once in a `Select`, so an upload retry reuses the same name while a later message or a replay gets a different one. Two candidates who both attach `Resume.pdf` cannot collide, and a new version cannot silently overwrite an earlier CV.

Corrected. The build summary still prints a line describing saves under a legacy `<Name>_<AppID>` shape with true in-place overwrite. That text is stale relative to the actions the same build generates, which use the GUID name. The reviewed edition's statement that the GUID overlay is present in the supplied package is the accurate one. Application-level versioning is not a storage retention lock, and an older deployed flow may still behave differently.

Historical. A separate naming defect was closed on 31 July 2026. Saves and the applicant-facing reference used to be built from the reference quoted in the message text, so a sender replying on a dead thread had their resume filed under a stale Application ID that P2 could never find. `AppRef_current` now reads the matched row's real Application ID and drives both the filename and the displayed reference. A one-time repair found 11 orphaned resume files on live SharePoint: 10 were byte-identical duplicates and were deleted, and one had genuinely newer content and was promoted. The follow-up check found zero orphans, zero missing files and zero duplicates.

Historical. `Last Updated Date` used to be stamped blindly from the incoming message time on all six patch actions, so replaying an older queued reply could move a row's timestamp backwards. Every patch now stamps the later of the existing value and the incoming time. This mattered downstream, because P2 reads that column to decide whether a candidate replied after being asked for missing information.

10.1 Attachment selection

`ResumeFilesAll` accepts PDF and DOCX. `ResumeFilesKept` then removes auxiliary documents by filename, covering portfolio, cover letter and its variants, transcript and similar terms. `ResumeFiles` falls back to the unfiltered list whenever filtering would leave nothing, so a candidate whose only attachment is called `portfolio.pdf` is still processed rather than rejected.

11. Failure semantics and resiliency

11.1 The never-block-the-queue principle, restated accurately

A malformed message must never stall intake for the applicants behind it. Cleanup is therefore deliberately unconditional: `Mark_as_read` and `Move_to_processed` hang off `HasResume` succeeding, not off the writes, and every action after a candidate-row patch lists `Failed` in its `runAfter`.

Corrected. The original TDD presented this as a guarantee that failures are always caught and always recoverable. It is better described as best-effort. Cleanup and alerting can themselves fail, several failures are absorbed by terminal actions, and the run still reports `Succeeded`. The cost of the design is exactly the silent-failure class that the 13 alerts exist to cover.

Historical, and the reason the alerts exist. `Mark_as_read` and `Move_to_processed` sit on a branch parallel to `Add_row`, so before August 2026 an `Add_row` failure produced no row, no reply, an archived message and a green run, in silence. The 24 August 2026 mailbox audit found 11 applicants lost exactly that way, clustered between 2 and 6 June 2026. Re-gating cleanup was rejected because it reintroduces the poisoned-message queue block and re-marking unread races the mutable Outlook message identifier. The alert is the signal, and its body carries the recovery step.

11.2 Retry policy

Action class	Intended policy	Where the policy is written in the package
Six applicant sends	none	<code>runtimeConfiguration.retryPolicy</code>
Fourteen admin notifications	exponential, count 2, PT10S	<code>runtimeConfiguration.retryPolicy</code>
Eleven mail moves	none	<code>inputs.retryPolicy</code>

The intent is sound. A retried applicant send can deliver two to four identical acknowledgments after a transient network drop, and a retried move races a mutable message identifier, while an admin alert is the only signal of a silent failure and must survive a blip.

Open, high severity. The package writes the same setting in two different places. All 11 `Movev2` actions carry `retryPolicy` inside `inputs`, which is where Microsoft's workflow definition language documents it. All 20 send actions, including every applicant-facing one, carry it under `runtimeConfiguration` instead. The current tests assert the builder's own placement, so a passing suite does not show that the connector honours the intended policy. Until an exported tenant definition is inspected and an ambiguous timeout is simulated, duplicate-send prevention is not established. This is a documented-schema mismatch; the active tenant's effective retry behaviour was not observed. Reference:

<https://learn.microsoft.com/en-us/azure/logic-apps/error-exception-handling> Evidence: `flow/build_zip.py` lines 1854-1871.

11.3 Other fallbacks that hold

The display-name fallback works as documented: the text before `<` when present, otherwise the local part with dots and underscores converted to spaces, and P2 later overrides the cell with the name parsed from the resume text. The attachment-filter bypass described in 10.1 holds. The destructive workbook self-heal is genuinely gone: a missing workbook now halts and alerts rather than re-uploading a blank template over live data.

12. Configuration reference

Neither TDD carried this. Edit `flow/flow_config.json`, run `python flow/build_zip.py`, then re-import the package. Nothing takes effect in the tenant until that import happens.

Key	Current value	Notes
flow_name	DriverAI_HiringAgent_P1_AutoReply	Package and flow display name
email.trigger_mailbox	apply@driverai.io	Watched Inbox and reply sender
email.admin_email	yashv@driverai.io	Alert recipient and BCC address
email.send_applicant_emails	false	Suppresses all six applicant sends; processing continues
email.send_admin_failure_alerts	true	Keeps all 13 failure alerts live regardless of the applicant posture
trigger.interval_min	1	1,440 runs per day; watch the request budget
trigger.unread_per_run	1	One message per sequential run
trigger.fetch_only_unread	false	Inbox membership is the queue, not the unread flag
business_rules.duplicate_check_days	90	Re-application window, inclusive at exactly 90 days
business_rules.duplicate_notice_max	5	Counter ceiling for the duplicate path
business_rules.update_resume_max	5	Counter ceiling for the update path
business_rules.followup_reply_max	5	Counter ceiling for the follow-up path
business_rules.reply_cap	3	How many contacts actually receive mail
sharepoint.site	https://led1234567.sharepoint.com/sites/CandidateList_HiringAgent	Site URL
sharepoint.resumes_folder	/Shared Documents/Candidate_Resumes	Library-qualified root for saves
sharepoint.documents_library_id	3600a6b3-142f-45f1-bce0-cfcb285c69e7	Required by CreateNewFolder
sharepoint.dated_resume_subfolders	true	Year and month foldering
excel.file	/Master_Files/Sharepoint_Master_File.xlsx	One workbook for all years
excel.table	HiringAgent_P1_Candidates	Table name
inbox_tidy.enabled	true	Move processed mail out of the Inbox
inbox_tidy.destination	Archive	Legitimate mail, marked read
inbox_tidy.spam_destination	Junk Email	Screened mail, left unread
inbox_tidy.failure_destination	Archive	Failed mail, left unread
appref.detect_pattern	app-20	Quoted-reference marker, valid to 2099

Key	Current value	Notes
appref.date_format, time_format, hex_length	yyyyMMdd, HHmm, 4	Reference token shape; the tail is derived from the message id, not a fresh GUID
year_separator.enabled	false	Live value; the README table still says true
month_separator.enabled	false	Live value; the README table still says true

Corrected. The README describes the reference tail as `toUpper(substring(guid(), 0, 4))`. The shipped `AppRefSeed` pads the message identifier and takes four characters from a fixed offset within it, so the reference is deterministic: replaying the same message mints the same Application ID. That is useful for recovery, because a replay after a failed `Add_row` recreates the row under its original reference. It does not make replay idempotent, since the stored filename still takes a fresh GUID.

Spam list sizes as shipped: 43 bad senders, 27 bad subjects, 123 content phrases across five groups, 12 link shorteners and 21 application keywords. To use a custom archive folder, resolve its identifier with a Graph call against `mailFolders` and set `inbox_tidy.destination` to that identifier.

What is not config-driven: the six subjects and HTML bodies, the row field map and the flow structure itself are Python in `build_zip.py`. The file `_pristine_send_actions.json` restores only the connector shape of those actions as a self-heal; it does not supply their text.

13. Operational runbook

13.1 Three rules

The Inbox is the to-do list, and mail sitting there is pending work. Archive means handled and Junk means silenced. A failure parks the message unread in Archive and alerts, rather than blocking the applicants behind it.

13.2 Replay and recovery

`trigger_reset.py` marks archived mail unread and moves it back to the Inbox, after which the ordinary poll processes one message per run. `Get_unread_emails` applies no date filter, so age is irrelevant and any archived message becomes live again the moment it returns unread.

Command	Effect
<code>python HiringAgent_P1/trigger_reset.py --dry-run</code>	List candidates for replay, change nothing
<code>python HiringAgent_P1/trigger_reset.py</code>	Replay the last 31 days
<code>python HiringAgent_P1/trigger_reset.py --days N</code>	Replay a custom window
<code>python HiringAgent_P1/trigger_reset.py --all</code>	Replay every message in Archive

Credentials come from `HiringAgent_App_P2/.env`, the same Entra application the P2 worker uses. It needs `Mail.ReadWrite` application permission with admin consent, in addition to the SharePoint permissions P2 already holds. A 403 on every mail-folder call is the signature of an Exchange Online application access policy that does not scope this application to the mailbox, not a missing Graph permission; that condition was present on 4 July 2026 and resolved by 15 July 2026. It never affected the flow itself, which authenticates through its own Power Automate connection. `bulk_move_tool.py` restores larger Archive batches the same way.

Open. Replay is not idempotent. A replayed message can create another file and consume another contact count, because P1 stores no stable source-message key. Neither tool can reach mail already stuck in the Inbox.

13.3 Investigating a candidate

Check the workbook row and its Mail Sent cell first. Because every applicant send carries a BCC to the admin address, a copy of any real outbound message exists in that mailbox. If no row exists, check Junk Email, since the message may have been screened. Remember that the run history holding the step-level detail expires after 30 days.

13.4 Deployment

1. Locate the package `DriverAI-Hiring-AutoReply-apply.zip`.
2. Open `make.powerautomate.com` in the DriverAI tenant, choose My flows, then Import and Import Package (Legacy).
3. Upload the ZIP and wait for inspection.
4. Set the flow name to `Update over DriverAI_HiringAgent_P1_AutoReply`, or Create as new, then map all three connections.
5. Import, then turn the flow on.

Corrected, and it matters more than it reads. The original TDD instructed the operator to map an Office 365 Outlook connection "authorized for `apply@driverai.io`". The package does not contain such a connection. All three connection resources in its manifest carry the display name `yashv@driverai.io`, so this is one user account holding all three, and applicant mail leaves the shared mailbox only because `SharedMailboxSendEmailV2` names `apply@driverai.io` as the mailbox address. That works while the mapped account keeps Send As or Send on Behalf rights on the shared mailbox, and stops working, or starts sending from the wrong identity, if those rights change or a different account maps the connection at import. The 14 admin notifications use the plain `SendEmailV2` operation instead, so they send from the mapped account's own mailbox rather than from the shared one.

Verified import behaviour. The package identifies the flow as 22f1cacc-4e05-41ca-806e-f4baf41de15b, created 18 June 2026, and its manifest sets suggestedCreationType to Update with all three connections marked Existing. A default import therefore overwrites the running flow rather than creating a copy. Combined with the enabled sends recorded above, an unattended Next-Next-Import is a live cutover, not a test.

Open, and the most important line in this section. The supplied package sends live applicant mail. flow_config.json and the loose definition.json both suppress it. Importing this ZIP does not reproduce local test settings, and does not match what the owner reports running. Before any import, confirm which posture is intended, and build into an explicit output path recording the mode, the config hash and the package hash.

14. Verification and what it proves

test_p1.py is a custom assertion script, run as `python test_p1.py` from the P1 directory. It is a structural and routing simulation. It parses the built package and the loose definition, checks action wiring, runAfter maps, column contracts, gate list sizes, timestamp expressions, BCC routing and alert coverage. It does not call any connector, so it cannot demonstrate an end-to-end send, an Excel write or a SharePoint upload.

Check performed this review	Result
Supplied ZIP against the current local config	691 passed, 17 failed
Isolated rebuild with applicant mail suppressed	708 passed, 0 failed
Isolated rebuild with applicant mail enabled	717 passed, 0 failed
Package identity across the attached and both workspace copies	Identical SHA-256
Static inspection of failure paths and templates	Section 15 findings stand despite the passing rebuilds

The 17 failures in the first row are the artifact-versus-config disagreement, not defects introduced by the review: the ZIP enables sends that the local config suppresses. The 717 figure comes from the enabled-mail rebuild, which is why it exceeds the suppressed build's 708 by exactly the nine send-specific assertions.

Rebuilds ran only in `tmp/p1_review_build`. No scoring, database reset, process restart, flow import, workbook upload or email send was performed. Logs are retained in `docs/review_20260908/`.

The honest reading. 717 passing structural checks establish that the package is internally consistent and correctly wired. They do not establish that a failed upload cannot be followed by a success acknowledgment, that the allow-list rejects a lookalike address, that the connector honours the no-retry setting, or that any message was ever delivered. Those four questions are exactly what section 15 records as open.

15. Consolidated findings register

15.1 High: the deployment artifact disagrees with the test configuration

The ZIP enables all six applicant sends to the incoming sender while `flow_config.json` and the loose definition suppress them. Importing it does not reproduce local test settings. Keep the artifact as evidence, build into an explicit output path with a recorded mode and both hashes, and reject any import artifact that disagrees with its reviewed configuration. No package was replaced during this review. Evidence: `flow/flow_config.json`, all six Send actions in the ZIP, `flow/build_zip.py` lines 2475-2505. The ZIP and the loose definition differ only in those six actions and four Mail Sent patches.

15.2 High: a success acknowledgment can follow a failed resume save

`IsUnderReplyCap_update` and `IsUnderReplyCap_dup` both run after their Save loop with Succeeded, Failed and Skipped accepted. The branch then sends copy stating that the application now reflects the latest version, and patches the row to New Email Received with the counter incremented, even though the new file was never stored. The admin alert fires but cannot unsend the applicant's message, and its own body says so: it warns that P2 will re-score the stale document and that the applicant has already been told their update was received.

New in this merge. The same class of defect exists on the new-application path, and neither earlier document recorded it. `Add_row` runs after `Save_resumes_to_SharePoint` with Failed and Skipped accepted, so a first-time applicant whose upload fails still gets a row, and `Send_acknowledgment`, gated on `Add_row` succeeding, still tells them the application was received. The row then points at a resume that does not exist.

The fix is the same in all three places: require committed persistence before success copy, and route a failed write into an explicit recoverable state. Fault-injection coverage is needed for each attachment and each sidecar. Evidence: `flow/build_zip.py` lines 1758-1766, `IsUnderReplyCap_dup`, `IsUnderReplyCap_update` and `Add_row` in the ZIP, `flow/reliability_flow.py` lines 78-86.

15.3 High: privileged tester matching is a substring check

`MatchAllowSender` evaluates `contains(LowerFrom, item())`. An address such as `yashv@driverai.io.example.net` contains the allow-listed string without being that mailbox, and a display name can contain it too. A match disables all three screening gates including the self-loop guard, and switches the reference lookup to Application ID selection, so this is considerably more than a spam false positive. Normalize the address and compare for equality, and keep tenant authentication separate from tester privilege. Evidence: `flow/build_zip.py` lines 449-466 and 1674-1679, and the three gate expressions in the ZIP, each carrying a `MatchAllowSender` length of zero as a conjunct.

15.4 High: the no-retry guarantee is not established by the serialized JSON

Covered in full in section 11.2. `_set_email` writes the policy under `runtimeConfiguration` while Microsoft documents it under `inputs`, and the same package uses the documented location for its 11 move actions. The tests assert the builder's own placement, so they cannot detect the mismatch.

15.5 High: P2's SQLite path has no delivery dispatcher

The isolated adapter refuses delivery and the parent pipeline never dispatches. Live P2 applicant mail needs a durable outbox after a successful result commit, retaining suppression and test-recipient routing, with idempotency and ambiguous-delivery reconciliation. This is separate from P1's templates. Evidence: `HiringAgent_App_P2/hiring_agent/local_pipeline.py` lines 76-79 and `run_local_pipeline` at lines 322-446; `sharepoint_scoring.py` branches there when the backend is sqlite.

15.6 Medium: text-only replies disappear from the candidate record

The follow-up patches store the counter, the update time and sometimes Mail Sent. They store neither the new Mail Body nor a reply sidecar. The archived email survives, but P2 cannot use typed clarifications through this contract. Either add a linked inbound-message event with explicit human or automated handling, or narrow the reply copy as suggested in 7.7. Evidence: `flow/build_zip.py` lines 1773-1816 and `_make_patch_item` at lines 1417-1456; `reliability_flow.py` creates sidecars only inside the resume-save loops.

15.7 Medium: multiple applications and replay lack deterministic selection

Covered in section 5. The latest timestamp decides eligibility while the first row receives the patch, and the external reference path also takes the first sender match. A legitimate sender with several applications can have the wrong one updated. Use a stable latest-eligible selection, or validate sender together with quoted identifier, and add a stable source-message replay key before automating recovery.

15.8 Medium: monitoring overstates what a successful run proves

`Notify_failure` watches `HasResume` only, not every preprocessing step. `Notify_ignored_mail` covers non-application mail but is not a processing-failure monitor. With best-effort cleanup, swallowed failures and terminal actions reporting success, neither a green run nor an empty Inbox is a health signal. Capture intake completion separately, and monitor queue age, orphaned files and sidecars, unscored age, import conflicts, mail outcomes and pending report publication.

15.9 Open items carried forward from the operations notes

No MIME, magic-byte or antivirus checking and no attachment size limit, as covered in section 4. No rate-limit or 429 backoff beyond the platform's own per-action retry. P1 is untracked in git, so nothing here is diffable against history. `trigger_reset.py` cannot recover mail already stuck in the Inbox.

16. What each source document was missing or got wrong

This is the merge delta. Every row is now resolved somewhere in sections 1 to 15.

Source	Gap or error	Where it is resolved here
Original TDD	No reply bodies, only subjects and purposes	Section 7
Original TDD	Spam group arithmetic summed to 111, and the 12 vendor-pitch phrases were undocumented	Section 4
Original TDD	Presented a 60-second SLA and zero Excel conflicts as guarantees	Section 3
Original TDD	Presented self-loop, virus screening, never-blocking and one-click replay as guarantees	Sections 4, 11 and 13.2
Original TDD	Described the allow-list as a testing convenience, not a privilege escalation path	Sections 4 and 15.3
Original TDD	Described the fake-reference guard without noting the allow-list bypass	Section 5
Original TDD	Off-by-one in the final-notice threshold	Section 5.1
Original TDD	An 11-versus-22 column split that hides the shared Mail Sent cell and sidecar-only values	Sections 2 and 9
Original TDD	No configuration reference, no observability table, no replay tooling	Sections 12, 8.1 and 13.2
Original TDD	No Rejected sheet contract	Section 9.1
Reviewed TDD	Located the false acknowledgment on the duplicate and update paths only	Section 15.2
Reviewed TDD	Deferred the exact column contract to other files	Section 9.1
Reviewed TDD	Did not carry the deployment steps, config reference or incident history	Sections 12, 13 and 11.1
Operations README	Column counts of 30 and 29, both stale	Section 9.1
Operations README	Bad-sender and bad-subject counts of 32 and 24, both stale	Section 4
Operations README	Separator flags recorded as true; the live config has them false	Section 12
Operations README	Nine retrying alerts and two intake-write alerts; there are 14 and 4	Section 8
Build summary output	Still prints a legacy filename shape the same build no longer generates	Section 10
All sources	No delivery ledger distinguishing attempted, suppressed, failed and delivered	Section 8.2
All sources	No replay key, no resumable write stages, no orphan reconciliation owner	Sections 9.2 and 13.2

17. Production acceptance still required

1. Export the active flow and compare it against the reviewed package. Record the effective P1 and P2 suppression posture, intended and actual recipients, and connection permissions. Keep test settings until a deliberate cutover.
2. Resolve findings 15.1 through 15.5, then rebuild reproducibly and verify both modes again. Do not treat a passing structural suite as the only launch gate.
3. In a controlled test mailbox, exercise all six reply types, the final-notice variant and the silent caps. Include HTML and display-name senders, a sender with multiple applications, a replayed message and a text-only clarification.
4. Inject upload, sidecar, Excel patch, send-timeout and Archive-move failures. Verify that no false acknowledgment is sent, no duplicate send occurs and no recovery evidence is lost.
5. Verify P2's full path: intake event, cached attachment hash, score commit, outbox disposition, both published reports and pending-publication recovery. Test restore and rollback before depending on unattended operation.

These are proposed acceptance steps, not actions performed. The running service, the current mail posture and the original documents were all left intact.

Appendix A: verified constants

Constant	Value
Flow display name	DriverAI_HiringAgent_P1_AutoReply
Package SHA-256	5fa60a5941bc60fbed3c41603169908684dd277bb77df10502f956888bb62711
Actions, top-level actions, maximum depth	131, 18, 8 of 8
Screening list sizes	43 senders, 27 subjects, 123 phrases, 12 shorteners, 21 keywords
Content phrase groups	30 scam, 36 offensive, 20 malware, 25 foreign-language, 12 vendor pitch
Applicant sends and admin notifications	6 and 14
Move actions carrying a no-retry policy	11
Workbook tables	33-column main, 32-column rejected
Fields written by Add_row	11
Duplicate window, contact caps, reply cap	90 days, 5, 3
Timezone offset applied to all derived dates	UTC minus 7